```
"""LogiX (PackOS) daily Tier 3 report extraction tool.

Automates the two manual exports described in the operations guideline
"Instruction for Export of LogiX Reports for Site Neustadt Tier 3 Dashboard":

  I.  Standort OEE Bericht  -> CSV   (all MUs/Lines except offline Line 66)
  II. Produktion / Bereich MU-Packaging -> XLSX (year / "Dieses Jahr")

Authentication is done on the main LogiX login page by filling the
"Login / Email" and "Password" fields and clicking the "Login" button.
When credentials are configured, the form is submitted automatically.
Otherwise the first run opens a visible browser so a human can complete
the sign-in once. The session is then persisted in a user-data directory
so subsequent runs reuse it without re-authenticating.

Usage
-----
    # First run (interactive login, headed browser):
    python logix.py --headed

    # Later runs (reuse stored session, can be headless):
    python logix.py

    # Reuse an EXISTING logged-in Edge/Chrome via CDP (no re-login / no MFA):
    #   1) start Edge with remote debugging:
    #        msedge --remote-debugging-port=9222 --user-data-dir=%TEMP%\\edge-deb
    #   2) attach:
    #        python logix.py --cdp-endpoint http://localhost:9222

    # Or launch an installed Edge instead of bundled Chromium:
    python logix.py --headed --channel msedge

Requirements
------------
    pip install playwright
    playwright install chromium
"""

from __future__ import annotations

import argparse
import datetime as _dt
import json
import os
```

```
import re
import shutil
import signal
import subprocess
import sys
import tempfile
import time
from pathlib import Path

from playwright.sync_api import (
    BrowserContext,
    Download,
    Page,
    TimeoutError as PWTimeoutError,
    sync_playwright,
)

# ---------------------------------------------------------------------------- #
# Configuration / defaults
# ---------------------------------------------------------------------------- #
DEFAULT_URL = "https://packos.logix.abbott.com/world"
DEFAULT_DOWNLOAD_DIR = Path.home() / "logix_exports"
DEFAULT_USER_DATA_DIR = Path.home() / ".logix_playwright_profile"

# Optional config file (config.json). It holds the "aws" settings block (see
# deliver.py) and may optionally hold "username"/"password" for LOCAL DEV only.
# In production credentials come from LOGIX_USERNAME / LOGIX_PASSWORD
# env vars or directly from AWS Secrets Manager via aws.secret_id in config.json;
# the file should not store real secrets.
# Looked up (in order): --config PATH, ./config.json, next to this script.
DEFAULT_CONFIG_NAME = "config.json"

# Site whose Tier 3 reports we export.
SITE_NAME = "Neustadt"

# How long (ms) to wait for slow navigations / report generation.
NAV_TIMEOUT = 60_000
ACTION_TIMEOUT = 30_000
# Report generation (export -> download) can take a while for large ranges.
DOWNLOAD_TIMEOUT = 180_000

# Areas (top-level checkboxes) that must be selected for the OEE export.
# Anything matching EXCLUDE_OEE is explicitly left unchecked (offline Line 66).
EXCLUDE_OEE = ("offline", "linie 66", "line 66")

# Checkboxes in the OEE dialog that are options, not MUs/lines: leave as-is.
SKIP_OEE_OPTIONS = ("exclude utilization", "utilization loss")
```

```
def log(msg: str) -> None:
    print(f"[logix] {msg}", flush=True)


def load_credentials(config_path: str | None) -> dict:
    """Load service-account credentials for unattended LogiX sign-in.

    Resolution order:
      1. username/password from config.json, if present (local dev only),
      2. LOGIX_USERNAME / LOGIX_PASSWORD environment variables,
      3. AWS Secrets Manager using config.json -> aws.secret_id.

    The production config.json already contains the AWS secret id under:
        {"aws": {"region": "...", "secret_id": "..."}}

    The AWS secret must contain JSON like:
        {"username": "...", "password": "..."}
    """
    candidates: list[Path] = []
    if config_path:
        candidates.append(Path(config_path).expanduser())
    candidates.append(Path.cwd() / DEFAULT_CONFIG_NAME)
    candidates.append(Path(__file__).resolve().parent / DEFAULT_CONFIG_NAME)

    config_data: dict = {}
    for path in candidates:
        if path.is_file():
            try:
                config_data = json.loads(path.read_text(encoding="utf-8"))
                log(f"Loaded config from {path}.")
                break
            except (json.JSONDecodeError, OSError) as exc:
                log(f"  ! could not read config {path} ({exc}); ignoring.")

    # 1) Optional local-dev credentials in config.json.
    creds: dict = {
        "username": (config_data.get("username") or "").strip(),
        "password": config_data.get("password") or "",
    }
    if creds.get("username") and creds.get("password"):
        log("Using username/password from config.json.")
        return creds

    # 2) Env vars exported by: eval "$(uv run deliver creds)".
    env_user = (os.environ.get("LOGIX_USERNAME") or "").strip()
```

```
env_pw = os.environ.get("LOGIX_PASSWORD") or ""
if env_user or env_pw:
    if env_user:
        creds["username"] = env_user
        log("Using username from LOGIX_USERNAME environment variable.")
    if env_pw:
        creds["password"] = env_pw
        log("Using password from LOGIX_PASSWORD environment variable.")
    if creds.get("username") and creds.get("password"):
        log("LogiX credentials are available for unattended login.")
        return creds
    log("  ! LOGIX_USERNAME/LOGIX_PASSWORD are not both set; continuing to AW

# 3) Direct AWS Secrets Manager fallback. This makes logix.py self-contained
# even if the caller forgot eval "$(uv run deliver creds)".
aws_cfg = config_data.get("aws") or {}
secret_id = (
    os.environ.get("LOGIX_SECRET_ID")
    or os.environ.get("LOGIX_SECRET_NAME")
    or aws_cfg.get("secret_id")
    or aws_cfg.get("logix_secret_id")
    or config_data.get("logix_secret_id")
)
region = (
    os.environ.get("AWS_REGION")
    or os.environ.get("AWS_DEFAULT_REGION")
    or aws_cfg.get("region")
)

if not secret_id:
    log("No LogiX credentials found and no AWS secret id configured (aws.secr
    return {}
if not region:
    log("No AWS region configured. Set aws.region in config.json or AWS_REGIO
    return {}

try:
    import boto3
    from botocore.exceptions import BotoCoreError, ClientError
except ImportError:
    log(
        "  ! boto3 is not installed, so AWS Secrets Manager cannot be used. "
        "Install with 'uv sync --extra aws' or run 'eval \"$(uv run deliver c
    )
    return {}

try:
```

```
        log(f"Reading LogiX credentials from AWS Secrets Manager secret '{secret_
        client = boto3.client("secretsmanager", region_name=region)
        response = client.get_secret_value(SecretId=secret_id)
        secret_string = response.get("SecretString")
        if not secret_string:
            log("  ! AWS secret has no SecretString.")
            return {}

        secret_data = json.loads(secret_string)
        username = (
            secret_data.get("username")
            or secret_data.get("LOGIX_USERNAME")
            or secret_data.get("login")
            or secret_data.get("email")
            or ""
        ).strip()
        password = secret_data.get("password") or secret_data.get("LOGIX_PASSWORD

        if username and password and username != "-" and password != "-":
            log("Loaded LogiX credentials from AWS Secrets Manager.")
            return {"username": username, "password": password}

        log(
            "  ! AWS secret was found, but it does not contain usable "
            "username/password keys."
        )
        return {}
    except (BotoCoreError, ClientError, json.JSONDecodeError, KeyError) as exc:
        log(f"  ! could not read LogiX credentials from AWS Secrets Manager: {exc
        return {}


# ---------------------------------------------------------------------------- #
# Authentication
# ---------------------------------------------------------------------------- #
def authenticate(page: Page, url: str, login_timeout: int, credentials: dict | No
    """Open the portal and ensure we end up on the authenticated dashboard.

    If the persisted session is still valid we land directly on the dashboard.
    Otherwise we stay on the main LogiX login page, enter credentials into the
    "Login / Email" and "Password" fields, and click the green "Login" button.
    This intentionally does NOT click the "Azure AD" button.
    """
    log(f"Navigating to {url}")
    page.goto(url, wait_until="domcontentloaded", timeout=NAV_TIMEOUT)

    if _dashboard_visible(page):
```

```
        log("Existing session detected - already authenticated.")
        ensure_english(page)
        return

    if credentials:
        _autofill_logix_login(page, credentials)
    else:
        log(
            "No credentials configured. Please enter Login / Email and Password "
            "on the LogiX login page, then click Login."
        )

    log(
        "Waiting for the LogiX dashboard to become visible "
        f"(up to {login_timeout // 1000}s)..."
    )
    _wait_for_dashboard(page, timeout=login_timeout)
    log("Authentication successful - dashboard is visible.")
    ensure_english(page)


def _autofill_logix_login(page: Page, credentials: dict) -> None:
    """Fill the main LogiX login form and click Login.

    This function uses two strategies:
      1. Playwright locators for normal forms.
      2. A JavaScript DOM fallback that finds visible input fields by type and
         screen position. This is useful for LogiX pages where the visible text
         is not a real HTML <label> associated with the input.
    """
    username = (credentials.get("username") or "").strip()
    password = credentials.get("password") or ""

    if not username or not password:
        log("  ! credentials object is missing username or password; cannot autof
        return

    log("Attempting unattended LogiX form sign-in with configured credentials..."

    # Strategy 1: normal Playwright locators.
    try:
        login_field = (
            page.locator("input[type='email']")
            .or_(page.locator("input[autocomplete='username']"))
            .or_(page.locator("input[placeholder*='Login' i]"))
            .or_(page.locator("input[placeholder*='Email' i]"))
            .or_(page.locator("input[name*='email' i]"))
```

```
            .or_(page.locator("input[name*='login' i]"))
            .or_(page.locator("input[name*='user' i]"))
            .or_(page.locator("input:not([type='hidden']):not([type='password'])"
        ).first
        login_field.wait_for(state="visible", timeout=10_000)
        login_field.click()
        login_field.fill(username)
        page.wait_for_timeout(300)

        password_field = (
            page.locator("input[type='password']")
            .or_(page.locator("input[autocomplete='current-password']"))
            .or_(page.locator("input[placeholder*='Password' i]"))
            .or_(page.locator("input[name*='password' i]"))
            .or_(page.locator("input[name*='passwd' i]"))
        ).first
        password_field.wait_for(state="visible", timeout=10_000)
        password_field.click()
        password_field.fill(password)
        page.wait_for_timeout(300)

        if _login_form_has_values(page):
            log("Filled LogiX login form using Playwright locators.")
        else:
            raise RuntimeError("Playwright locators ran but fields still appear e
    except Exception as exc:  # noqa: BLE001
        log(f"  ! normal locator fill did not work ({exc}); trying DOM fallback."
        try:
            ok = _fill_logix_login_via_dom(page, username, password)
            if not ok:
                log("  ! DOM fallback could not find/fill the login form.")
                _log_login_form_diagnostics(page)
                return
            log("Filled LogiX login form using DOM fallback.")
        except Exception as dom_exc:  # noqa: BLE001
            log(f"  ! DOM fallback failed ({dom_exc}).")
            _log_login_form_diagnostics(page)
            return

    try:
        login_button = (
            page.get_by_role("button", name=re.compile(r"^\s*login\s*$", re.I))
            .or_(page.locator("button:has-text('Login')"))
            .or_(page.locator("input[type='submit'][value*='Login' i]"))
            .or_(page.locator("button[type='submit'], input[type='submit']"))
        ).first
        login_button.wait_for(state="visible", timeout=10_000)
```

```
        login_button.click()
        log("Clicked the main LogiX Login button.")
    except Exception as exc:  # noqa: BLE001
        log(f"  ! normal Login button click did not work ({exc}); trying DOM clic
        try:
            clicked = page.evaluate(
                """
                () => {
                  const visible = el => {
                    const r = el.getBoundingClientRect();
                    const s = window.getComputedStyle(el);
                    return r.width > 0 && r.height > 0 && s.visibility !== 'hidde
                  };
                  const controls = [...document.querySelectorAll('button,input[ty
                  const login = controls.find(el => /login/i.test((el.innerText |
                            || controls.find(el => (el.type || '').toLowerCase()
                  if (login) { login.click(); return true; }
                  const form = document.querySelector('form');
                  if (form) { form.requestSubmit ? form.requestSubmit() : form.su
                  return false;
                }
                """
            )
            if clicked:
                log("Clicked/submitted the LogiX login form using DOM fallback.")
            else:
                log("  ! could not find a Login button/form to submit.")
        except Exception as dom_exc:  # noqa: BLE001
            log(f"  ! DOM Login button fallback failed ({dom_exc}).")


def _login_form_has_values(page: Page) -> bool:
    """Return True if visible username and password inputs currently have values.
    try:
        return bool(page.evaluate(
            """
            () => {
              const visible = el => {
                const r = el.getBoundingClientRect();
                const s = window.getComputedStyle(el);
                return r.width > 0 && r.height > 0 && s.visibility !== 'hidden' &
              };
              const inputs = [...document.querySelectorAll('input')].filter(visib
              const user = inputs.find(i => (i.type || '').toLowerCase() !== 'pas
              const pass = inputs.find(i => (i.type || '').toLowerCase() === 'pas
              return !!(user && user.value && pass && pass.value);
            }
```

```
            """
        ))
    except Exception:  # noqa: BLE001
        return False


def _fill_logix_login_via_dom(page: Page, username: str, password: str) -> bool:
    """Fill visible LogiX username/password inputs with JavaScript."""
    return bool(page.evaluate(
        """
        ({ username, password }) => {
          const visible = el => {
            const r = el.getBoundingClientRect();
            const s = window.getComputedStyle(el);
            return r.width > 0 && r.height > 0 && s.visibility !== 'hidden' && s.
          };
          const setValue = (el, value) => {
            const proto = el instanceof HTMLTextAreaElement ? HTMLTextAreaElement
            const setter = Object.getOwnPropertyDescriptor(proto, 'value')?.set;
            if (setter) setter.call(el, value); else el.value = value;
            el.dispatchEvent(new Event('input', { bubbles: true }));
            el.dispatchEvent(new Event('change', { bubbles: true }));
            el.dispatchEvent(new Event('blur', { bubbles: true }));
          };

          const inputs = [...document.querySelectorAll('input')].filter(visible);
          const passwordInput = inputs.find(i => (i.type || '').toLowerCase() ===
          const usernameInput = inputs.find(i => {
              const t = (i.type || 'text').toLowerCase();
              const attrs = `${i.name || ''} ${i.id || ''} ${i.placeholder || ''}
              return i !== passwordInput && t !== 'hidden' && /email|login|user|t
            }) || inputs.find(i => i !== passwordInput && (i.type || '').toLowerC

          if (!usernameInput || !passwordInput) return false;
          usernameInput.focus();
          setValue(usernameInput, username);
          passwordInput.focus();
          setValue(passwordInput, password);
          return !!(usernameInput.value && passwordInput.value);
        }
        """,
        {"username": username, "password": password},
    ))


def _log_login_form_diagnostics(page: Page) -> None:
    """Log non-secret diagnostics about visible inputs/buttons on the login page.
```

```
    try:
        diag = page.evaluate(
            """
            () => {
              const visible = el => {
                const r = el.getBoundingClientRect();
                const s = window.getComputedStyle(el);
                return r.width > 0 && r.height > 0 && s.visibility !== 'hidden' &
              };
              const inputs = [...document.querySelectorAll('input')].filter(visib
                type: i.type || '', name: i.name || '', id: i.id || '', placehold
                autocomplete: i.autocomplete || '', hasValue: !!i.value
              }));
              const buttons = [...document.querySelectorAll('button,input[type=su
                ((b.innerText || b.value || b.textContent || '').trim()).slice(0,
              ).filter(Boolean);
              return { url: location.href, title: document.title, inputs, buttons
            }
            """
        )
        log(f"Login form diagnostics: {diag}")
    except Exception as exc:  # noqa: BLE001
        log(f"  ! could not collect login form diagnostics ({exc}).")


def ensure_english(page: Page) -> None:
    """Make sure the UI language is English ("EN").

    The header shows a language indicator (``div.text-uppercase.text-white``)
    whose text is the current language code ("EN"/"DE"). If it is not already
    English we open the dropdown and click "English". This keeps all later
    label matching (tabs, buttons, etc.) consistent.
    """
    try:
        indicator = page.locator("div.text-uppercase.text-white").first
        indicator.wait_for(state="visible", timeout=10_000)
    except PWTimeoutError:
        log("Language indicator not found - assuming English.")
        return

    current = (indicator.inner_text() or "").strip().upper()
    if current == "EN":
        log("UI language already set to EN.")
        return

    log(f"UI language is '{current}' - switching to English...")
    try:
```

```
        indicator.click()
        page.wait_for_timeout(800)
        page.get_by_text(re.compile(r"^\s*English\s*$", re.I)).first.click(
            timeout=ACTION_TIMEOUT
        )
        page.wait_for_timeout(1_500)
        log("UI language switched to English.")
    except Exception as exc:  # noqa: BLE001
        log(f"  ! could not switch language to English ({exc}).")


def _dashboard_visible(page: Page) -> bool:
    """Heuristic: after login the PackOS 'world' view shows site cards
    (e.g. the Neustadt card) and a left nav with 'Reports'."""
    try:
        page.get_by_role("heading", name=SITE_NAME, exact=True).first.or_(
            page.get_by_text("Reports", exact=False)
        ).first.wait_for(state="visible", timeout=5_000)
        return True
    except PWTimeoutError:
        return False


def _wait_for_dashboard(page: Page, timeout: int) -> None:
    page.get_by_role("heading", name=SITE_NAME, exact=True).first.or_(
        page.get_by_text("Reports", exact=False)
    ).first.wait_for(state="visible", timeout=timeout)


# ---------------------------------------------------------------------------- #
# Shared navigation helpers
# ---------------------------------------------------------------------------- #
def select_site(page: Page, site: str = SITE_NAME) -> None:
    """From the 'world' overview, open the site by clicking its card.

    The world view renders one ``div.card.hoverable`` per site plus a
    non-clickable map marker (``div.plant-marker``) that also contains the
    site name. We must click the *card's heading* - clicking the plain text
    can hit the map marker, which does nothing.
    """
    # If we are already inside a site (Reports nav present), skip.
    if "/reports" in page.url or "/dashboard" in page.url:
        return
    log(f"Selecting site '{site}' from the world overview...")
    card = page.locator("div.card.hoverable").filter(
        has=page.get_by_role("heading", name=site, exact=True)
    )
```

```
    try:
        card.first.wait_for(state="visible", timeout=ACTION_TIMEOUT)
        card.first.click()
    except PWTimeoutError:
        # Fall back to clicking the heading directly.
        page.get_by_role("heading", name=site, exact=True).first.click()
    page.wait_for_url("**/dashboard/**", timeout=NAV_TIMEOUT)
    page.wait_for_timeout(1_000)
    log(f"Entered site '{site}' ({page.url}).")


def open_reports(page: Page) -> None:
    """Ensure we are on the site's Reports page.

    If we are already somewhere under ``/reports/`` (e.g. after a previous
    export), there is nothing to do. Otherwise click the left-nav 'Reports'
    item and wait for the navigation.
    """
    if "/reports/" in page.url:
        return
    select_site(page)
    log("Opening 'Reports' (left nav)...")
    page.get_by_text("Reports", exact=False).first.click()
    try:
        page.wait_for_url("**/reports/**", timeout=NAV_TIMEOUT)
    except PWTimeoutError:
        # Already on a reports URL or SPA route didn't trigger a load event.
        if "/reports/" not in page.url:
            raise
    page.wait_for_timeout(800)


def _click_tab(page: Page, label: str) -> None:
    """Click one of the report tabs (CUSTOM/OEE/PRODUCTION/REPORTS DOWNLOAD...).

    The tab captions are uppercased by CSS but the DOM text is normal case,
    so we match case-insensitively.
    """
    pattern = re.compile(rf"^\s*{re.escape(label)}\s*$", re.I)
    page.get_by_text(pattern).first.click()
    page.wait_for_timeout(1_000)


def _save_download(download: Download, download_dir: Path) -> Path:
    download_dir.mkdir(parents=True, exist_ok=True)
    suggested = download.suggested_filename or "logix_export"
    stamp = _dt.datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
    target = download_dir / f"{stamp}_{suggested}"
    download.save_as(target)
    log(f"Saved download -> {target}")
    return target


# ---------------------------------------------------------------------------- #
# Export I: Standort OEE Bericht (CSV)
# ---------------------------------------------------------------------------- #
def export_oee(page: Page, download_dir: Path, year: int) -> Path:
    """Export the Site OEE Report for all MUs except offline lines (CSV)."""
    log("=== Export I: Site OEE Report (CSV) ===")
    open_reports(page)

    log("Opening the 'REPORTS DOWNLOAD' tab...")
    _click_tab(page, "REPORTS DOWNLOAD")

    log("Opening 'Site OEE Report'...")
    page.get_by_text("Site OEE Report", exact=False).first.click()

    # The configuration dialog opens.
    dialog = page.get_by_role("dialog").filter(has_text="Site OEE Report")
    try:
        dialog.wait_for(state="visible", timeout=ACTION_TIMEOUT)
        scope = dialog
    except PWTimeoutError:
        # Fall back to the whole page if the dialog has no ARIA role.
        scope = page

    _select_oee_areas(scope)
    _set_periode(scope, "Custom")
    _set_oee_date_range(page, scope, year)

    log("Clicking 'Export report' and waiting for CSV download...")
    export_btn = scope.get_by_role(
        "button", name=re.compile(r"Export report", re.I)
    ).or_(scope.get_by_text(re.compile(r"Export report", re.I)))
    with page.expect_download(timeout=DOWNLOAD_TIMEOUT) as dl_info:
        export_btn.first.click()
    saved = _save_download(dl_info.value, download_dir)
    # Close the dialog so a following export is not blocked by the overlay.
    _close_dialog(page)
    return saved


def _close_dialog(page: Page) -> None:
    """Dismiss an open modal dialog (× button, else Escape)."""
```

```
    try:
        close = page.get_by_role("dialog").get_by_role(
            "button", name=re.compile(r"^\s*[×x]\s*$|close", re.I)
        )
        if close.count() > 0:
            close.first.click(timeout=3_000)
        else:
            page.keyboard.press("Escape")
    except Exception:  # noqa: BLE001
        try:
            page.keyboard.press("Escape")
        except Exception:  # noqa: BLE001
            pass
    page.wait_for_timeout(600)


class IncompleteSelectionError(RuntimeError):
    """Raised when one or more MU/line checkboxes couldn't be toggled.

    Previously these failures were only logged, so the export would proceed
    silently missing whichever MUs failed to check - producing an
    incomplete-looking report with no visible error. Raising here instead
    makes the run fail loudly (and, combined with the retry loop in run(),
    gives it a fresh browser context to try again automatically).
    """


def _select_oee_areas(scope) -> None:
    """Check every MU/area checkbox except the offline lines / Line 66.

    The dialog also contains option checkboxes (e.g. "Exclude utilization
    loss") that are NOT MUs - those are left at their default state. Toggling
    is done by clicking the associated <label> because the raw <input> is
    visually covered by its label and cannot be clicked directly.
    """
    log("Selecting all MUs/Lines (excluding offline lines / Linie 66)...")
    checkboxes = scope.get_by_role("checkbox")
    count = checkboxes.count()
    failed: list[str] = []
    for i in range(count):
        cb = checkboxes.nth(i)
        label = (_checkbox_label(cb) or "").strip()
        low = label.lower()
        if any(token in low for token in SKIP_OEE_OPTIONS):
            continue  # not an MU/line - leave at default
        want_checked = not any(token in low for token in EXCLUDE_OEE)
        try:
```

```
            if cb.is_checked() == want_checked:
                continue
            _toggle_checkbox(scope, cb)
            # Verify the click actually registered - a mis-timed click can
            # silently no-op if the dialog was still animating/loading.
            if cb.is_checked() != want_checked:
                raise RuntimeError("checkbox state unchanged after click")
        except Exception as exc:  # noqa: BLE001
            failed.append(label or f"checkbox #{i}")
            log(f"  ! could not toggle checkbox '{label}': {exc}")

    if failed:
        raise IncompleteSelectionError(
            f"Failed to toggle {len(failed)} MU/line checkbox(es), export "
            f"would be incomplete: {', '.join(failed)}"
        )


def _toggle_checkbox(scope, cb) -> None:
    """Click a checkbox via its <label for=id> (input is overlaid by label)."""
    cb_id = cb.get_attribute("id")
    if cb_id:
        label_loc = scope.locator(f"label[for='{cb_id}']")
        if label_loc.count() > 0:
            label_loc.first.click()
            return
    cb.click(force=True)


def _checkbox_label(checkbox) -> str | None:
    """Best-effort retrieval of a checkbox's visible label text."""
    # Prefer the linked <label for=id> text.
    try:
        text = checkbox.evaluate(
            "el => { const l = el.id && document.querySelector(`label[for='${el.i
            " return (l || el.closest('label') || el.parentElement)?.innerText ||
        )
        if text and text.strip():
            return text
    except Exception:  # noqa: BLE001
        pass
    for attr in ("aria-label", "name", "value"):
        val = checkbox.get_attribute(attr)
        if val:
            return val
    return None
```

```
# ---------------------------------------------------------------------------- #
# Export II: Produktion / Bereich MU-Packaging (XLSX)
# ---------------------------------------------------------------------------- #
def export_production_yield(page: Page, download_dir: Path) -> Path:
    """Export packaging-department production yield for the year (XLSX)."""
    log("=== Export II: Production - Bereich MU-Packaging (XLSX) ===")
    open_reports(page)

    log("Opening the 'PRODUCTION' tab...")
    _click_tab(page, "PRODUCTION")
    page.wait_for_timeout(800)

    _select_packaging_scope(page)

    # Selecting Period = 'Year' automatically sets Moment = 'This year'
    # (URL becomes period=5&moment=thisYear), which is exactly the guideline's
    # "Dieses Jahr". No separate Moment step is required.
    _set_period_dropdown(page, "Year")
    _ensure_moment_this_year(page)
    page.wait_for_timeout(1_000)

    log("Opening the report (table) view...")
    _open_report_table(page)

    log("Clicking 'Excel' and waiting for XLSX download...")
    with page.expect_download(timeout=DOWNLOAD_TIMEOUT) as dl_info:
        page.get_by_role("button", name="Excel", exact=False).or_(
            page.get_by_text("Excel", exact=False)
        ).first.click()
    return _save_download(dl_info.value, download_dir)


def _select_packaging_scope(page: Page) -> None:
    """Open the scope selector and pick Neustadt > Bereich MU-Packaging.

    The content-area scope selector is the ``.underscore-select`` element
    (an ``<h5>`` showing the current node, e.g. "Neustadt"). Clicking it opens
    a dropdown with a search box and the taxonomy tree. We type the area name
    and click the match. Note this is NOT the blue header site-switcher
    (``.plant-title``).
    """
    log("Selecting scope 'Neustadt' > 'Bereich MU-Packaging'...")
    page.locator(".underscore-select").first.click()
    page.wait_for_timeout(800)
    # Use the dropdown's search box to filter the tree, then click the node.
    try:
```

```
        search = page.get_by_placeholder("Search", exact=False)
        search.first.fill("Bereich MU-Packaging")
        page.wait_for_timeout(800)
    except Exception:  # noqa: BLE001
        pass
    page.get_by_text("Bereich MU-Packaging", exact=True).first.click()
    page.wait_for_timeout(1_000)


def _open_report_table(page: Page) -> None:
    """Click the report/table icon (``.report-data``) that opens the data grid.

    The opened modal contains the Excel/Print export buttons.
    """
    candidates = [
        page.locator(".report-data"),
        page.locator("button.icon-btn:has(svg)"),
        page.get_by_role("button", name=re.compile(r"report", re.I)),
    ]
    for cand in candidates:
        try:
            if cand.count() > 0:
                cand.first.click()
                page.get_by_text("Excel", exact=False).first.wait_for(
                    state="visible", timeout=ACTION_TIMEOUT
                )
                return
        except PWTimeoutError:
            continue
        except Exception:  # noqa: BLE001
            continue
    raise RuntimeError("Could not open the Production report table modal.")


# ---------------------------------------------------------------------------- #
# Period / moment selectors
# ---------------------------------------------------------------------------- #
def _set_periode(scope, value: str) -> None:
    """Set the OEE 'Period' <select> (Shift/Day/Week/Month/Custom)."""
    log(f"Setting Period = '{value}'...")
    try:
        scope.locator(
            f"select:has(option:text-is('{value}'))"
        ).first.select_option(label=value, timeout=5_000)
        return
    except Exception:  # noqa: BLE001
        pass
```

```
    # Fallback: first <select> on the dialog.
    try:
        scope.locator("select").first.select_option(label=value, timeout=5_000)
    except Exception as exc:  # noqa: BLE001
        log(f"  ! could not set Period='{value}': {exc}")


def _set_period_dropdown(page: Page, value: str) -> None:
    """Set the Production 'Period' button-dropdown (e.g. 'Year').

    The control is a button showing the current selection (default 'Day').
    Clicking the current value opens the option list; then click the target.

    Verifies the change actually took effect afterwards. A mis-timed or
    mistargeted click here previously failed silently: the export would
    then run against whatever period was already selected (usually the
    default 'Day'), producing a single day of hourly data with no error at
    all - which is why exports could look complete but only cover today.
    """
    log(f"Setting Period = '{value}'...")
    trigger = None
    for cur in ("Day", "Shift", "Week", "Month", "Last days"):
        try:
            cand = page.get_by_text(re.compile(rf"^\s*{cur}\s*$")).first
            cand.click(timeout=2_500)
            trigger = cand
            break
        except Exception:  # noqa: BLE001
            continue
    if trigger is None:
        try:
            trigger = page.locator(
                "xpath=//*[normalize-space(.)='Period']/following::*"
                "[self::button or self::div][1]"
            ).first
            trigger.click(timeout=4_000)
        except Exception:  # noqa: BLE001
            trigger = None

    page.wait_for_timeout(700)
    page.get_by_text(re.compile(rf"^\s*{re.escape(value)}\s*$")).first.click(
        timeout=ACTION_TIMEOUT
    )
    page.wait_for_timeout(1_000)

    current = ""
    if trigger is not None:
```

```
        try:
            current = (trigger.text_content(timeout=2_000) or "").strip()
        except Exception:  # noqa: BLE001
            current = ""
    if current.strip().lower() != value.strip().lower():
        raise RuntimeError(
            f"Period selector still shows '{current or '?'}' after trying to "
            f"set it to '{value}' - the export would have run against the "
            f"wrong (default) date range instead of failing loudly."
        )


def _ensure_moment_this_year(page: Page) -> None:
    """Best-effort: confirm Moment = 'This year' (auto-set by Period=Year).

    If for some reason it is not, open the Moment dropdown and pick it.
    """
    if "moment=thisyear" in page.url.lower():
        return
    log("Ensuring Moment = 'This year'...")
    try:
        page.locator(
            "xpath=//*[normalize-space(.)='Moment']/following::*"
            "[self::button or self::div][1]"
        ).first.click(timeout=4_000)
        page.wait_for_timeout(600)
        page.get_by_text(re.compile(r"^\s*This year\s*$", re.I)).first.click(
            timeout=4_000
        )
        page.wait_for_timeout(800)
    except Exception as exc:  # noqa: BLE001
        log(f"  ! could not set Moment='This year' ({exc}); using current value."


def _set_oee_date_range(page: Page, scope, year: int) -> None:
    """Set the OEE custom date range: 01.01.<year> .. today via the calendar.

    Selecting 'Custom' as the Period reveals a date-range field showing e.g.
    "09.06.2026 - 10.06.2026". Clicking it opens a dual-month calendar:
    the left calendar sets the start date, the right one the end date.
    """
    end = _dt.date.today()
    start = _dt.date(year, 1, 1)
    log(f"Setting OEE date range = {start:%d.%m.%Y} .. {end:%d.%m.%Y} ...")

    # Open the date-range picker by clicking the field that shows "dd.mm.yyyy".
    try:
```

```
        scope.get_by_text(re.compile(r"\d{2}\.\d{2}\.\d{4}\s*-\s*\d{2}\.\d{2}\.\d
                          ).first.click(timeout=5_000)
    except Exception:  # noqa: BLE001
        # Try a generic date-looking cell.
        scope.get_by_text(re.compile(r"\d{2}\.\d{2}\.\d{4}")).first.click()
    page.wait_for_timeout(800)

    calendars = page.locator(".calendar.lm-w-100")
    if calendars.count() < 1:
        log("  ! calendar widget not found; leaving default range.")
        return
    left = calendars.nth(0)
    right = calendars.nth(1) if calendars.count() > 1 else calendars.nth(0)

    _calendar_pick(page, left, start)
    _calendar_pick(page, right, end)
    page.wait_for_timeout(500)


_MONTHS_EN = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December",
]


def _calendar_month_index(label, month_re, year_re) -> int | None:
    """Return year*12+month for the calendar's current header, or None."""
    raw = label.first.inner_text() or ""
    m = month_re.search(raw)
    y = year_re.search(raw)
    if not m or not y:
        return None
    cur_m = _MONTHS_EN.index(m.group(1).capitalize()) + 1
    return int(y.group(0)) * 12 + cur_m


def _calendar_pick(page: Page, calendar, target: _dt.date) -> None:
    """Navigate one calendar to target's month/year and click the day.

    The month/year header (``.datepicker-container-label``) may contain extra
    whitespace/markup, so we extract the month name and 4-digit year with a
    regex and compare numerically. After each arrow click we wait until the
    header actually changes before deciding the next move (the re-render can
    lag, which previously caused premature stops).
    """
    want_idx = target.year * 12 + target.month
    label = calendar.locator(".datepicker-container-label")
```

```
    month_re = re.compile(
        r"(January|February|March|April|May|June|July|August|"
        r"September|October|November|December)",
        re.I,
    )
    year_re = re.compile(r"(19|20)\d{2}")

    for _ in range(120):
        cur_idx = _calendar_month_index(label, month_re, year_re)
        if cur_idx is None or cur_idx == want_idx:
            break
        arrow = ".datepicker-prev" if cur_idx > want_idx else ".datepicker-next"
        calendar.locator(arrow).first.click()
        # Wait until the header changes (up to ~3s) before the next decision.
        changed = False
        for _ in range(30):
            page.wait_for_timeout(100)
            if _calendar_month_index(label, month_re, year_re) != cur_idx:
                changed = True
                break
        if not changed:
            break  # arrow had no effect; avoid an infinite loop

    # Click the day number within this calendar's day grid (exact match avoids
    # hitting e.g. "1" inside "11"/"21").
    day_cell = calendar.locator(
        ".datepicker-cell, .datepicker-day, td, span, button"
    ).filter(has_text=re.compile(rf"^\s*{target.day}\s*$"))
    if day_cell.count() == 0:
        day_cell = calendar.get_by_text(str(target.day), exact=True)
    day_cell.first.click()
    page.wait_for_timeout(300)


# ---------------------------------------------------------------------------- #
# Browser acquisition
# ---------------------------------------------------------------------------- #
def _purge_previous_sessions(persistent_dir: Path) -> None:
    """Delete previously stored login sessions before an ephemeral run.

    Removes the persistent browser profile (default ``~/.logix_playwright_profile
    or whatever ``--user-data-dir`` points at) plus any leftover temporary
    profiles from earlier ephemeral runs that did not clean up (e.g. crashes).
    Best-effort: never raises.
    """
    persistent_dir = persistent_dir.expanduser()
    if persistent_dir.exists():
```

```
        shutil.rmtree(persistent_dir, ignore_errors=True)
        log(f"Ephemeral session: deleted previous profile {persistent_dir}.")

    # Sweep stale temp profiles from prior ephemeral runs.
    for leftover in Path(tempfile.gettempdir()).glob("logix_session_*"):
        if leftover.is_dir():
            shutil.rmtree(leftover, ignore_errors=True)
            log(f"Ephemeral session: removed stale temp profile {leftover}.")


def _kill_profile_processes(user_data_dir: Path) -> None:
    """Hard-kill any browser process bound to ``user_data_dir``.

    Used to close an ephemeral (throwaway) session instantly instead of waiting
    for Chromium's graceful profile flush. Matches the unique profile path on
    the process command line. Best-effort: never raises.
    """
    target = str(user_data_dir)
    try:
        out = subprocess.run(
            ["pgrep", "-f", target],
            capture_output=True,
            text=True,
            check=False,
        )
    except (OSError, ValueError):
        return
    for line in out.stdout.split():
        try:
            os.kill(int(line), signal.SIGKILL)
        except (ProcessLookupError, ValueError, PermissionError):
            pass


def _make_context(pw, args, download_dir: Path):
    """Return a (context, page, closer) tuple.

    Two strategies are supported:

    1. ``--cdp-endpoint`` -> attach to an ALREADY RUNNING browser (Edge/Chrome)
       that was started with remote debugging, e.g.::

            # Windows / Edge
            msedge.exe --remote-debugging-port=9222 --user-data-dir="%TEMP%\\edge-
            # then:
            python logix.py --cdp-endpoint http://localhost:9222
```

```
   This reuses your existing, already-authenticated session - no re-login
   and no MFA - which is exactly the benefit of an MCP/CDP browser link.

2. Default -> launch Playwright's own Chromium with a PERSISTENT profile so
   the first interactive login is remembered for all later (headless,
   scheduleable) runs. Works on headless servers with no local browser.
"""
if args.cdp_endpoint:
    log(f"Attaching to existing browser via CDP: {args.cdp_endpoint}")
    browser = pw.chromium.connect_over_cdp(args.cdp_endpoint)
    context = browser.contexts[0] if browser.contexts else browser.new_contex
        accept_downloads=True
    )
    context.set_default_timeout(ACTION_TIMEOUT)
    context.set_default_navigation_timeout(NAV_TIMEOUT)
    page = context.pages[0] if context.pages else context.new_page()

    def closer() -> None:
        # Do NOT close the user's browser; just detach.
        browser.close()

    return context, page, closer

user_data_dir = Path(args.user_data_dir).expanduser()
ephemeral = bool(getattr(args, "ephemeral_session", False))
if ephemeral:
    # First, purge any PREVIOUS sessions so nothing stale lingers:
    #   - the persistent profile (default or --user-data-dir), and
    #   - leftover temp profiles from earlier crashed ephemeral runs.
    _purge_previous_sessions(user_data_dir)
    # Then use a fresh throwaway profile that is deleted again on exit.
    user_data_dir = Path(tempfile.mkdtemp(prefix="logix_session_"))
    log("Ephemeral session: using a fresh profile (deleted on exit).")
user_data_dir.mkdir(parents=True, exist_ok=True)
log(
    f"Launching {'Edge' if args.channel else 'Chromium'} "
    f"({'headed' if args.headed else 'headless'}), profile={user_data_dir}"
)
# Flags that make headed Chromium render reliably on remote/virtual
# displays (WSLg, VNC, X-forwarding, containers). Without these the window
# often opens but the page area stays blank because Chromium tries to use
# a GPU/compositor that is not really available. Forcing software
# rendering (SwiftShader via ANGLE) fixes the blank-window symptom.
browser_args = [
    "--window-size=1920,1080",
    "--disable-gpu",
    "--use-gl=angle",
```

```
    "--use-angle=swiftshader",
    "--disable-software-rasterizer",
    "--disable-dev-shm-usage",
    "--no-sandbox",
    "--no-first-run",
]
launch_kwargs = dict(
    user_data_dir=str(user_data_dir),
    headless=not args.headed,
    accept_downloads=True,
    args=browser_args,
    viewport={"width": 1920, "height": 1080},
)
if args.channel:
    # Use an installed branded browser (e.g. 'msedge' or 'chrome').
    launch_kwargs["channel"] = args.channel
context: BrowserContext = pw.chromium.launch_persistent_context(**launch_kwar
context.set_default_timeout(ACTION_TIMEOUT)
context.set_default_navigation_timeout(NAV_TIMEOUT)
page = context.pages[0] if context.pages else context.new_page()

def closer() -> None:
    if ephemeral:
        # Throwaway profile: there is nothing worth flushing, so hard-kill
        # the browser process(es) bound to this unique profile dir. This
        # closes the window immediately instead of waiting several seconds
        # for Chromium's graceful profile flush.
        log("Closing browser...")
        _kill_profile_processes(user_data_dir)
        try:
            context.close()  # returns quickly; the browser is already gone
        except Exception:  # noqa: BLE001
            pass
        shutil.rmtree(user_data_dir, ignore_errors=True)
        log(f"Ephemeral session: deleted profile {user_data_dir}.")
        return

    # Persistent profile: close gracefully so the saved login/session is
    # written out cleanly (a few seconds in headed mode). Close pages first
    # so lingering dialogs/downloads don't block the shutdown.
    log("Closing browser (saving session; can take a few seconds)...")
    browser = context.browser
    try:
        for pg in list(context.pages):
            try:
                pg.close()
            except Exception:  # noqa: BLE001
```

```
                    pass
            context.close()
        except Exception:  # noqa: BLE001
            pass
        if browser is not None:
            try:
                browser.close()
            except Exception:  # noqa: BLE001
                pass

    return context, page, closer


# ---------------------------------------------------------------------------- #
# Orchestration
# ---------------------------------------------------------------------------- #
def _upload_to_s3(results: list[Path], args: argparse.Namespace) -> None:
    """Upload the freshly extracted files to S3 via the deliver module.

    Kept as a thin, lazily-imported bridge so logix.py has no hard dependency
    on boto3 unless --s3 is actually used.
    """
    try:
        import deliver
    except ImportError:
        log(
            "ERROR: --s3 requested but boto3/deliver is unavailable. "
            "Install the AWS extra: uv sync --extra aws"
        )
        raise

    aws_cfg = deliver.load_aws_config(args.config)
    bucket = args.s3_bucket or aws_cfg["bucket"]
    prefix = args.s3_prefix or aws_cfg["prefix"]
    region = args.s3_region or aws_cfg["region"]
    log(f"Uploading {len(results)} file(s) to s3://{bucket}/{prefix}/ ...")
    deliver.upload_files(results, bucket=bucket, prefix=prefix, region=region)


def _upload_error_screenshot(shot: Path, args: argparse.Namespace) -> None:
    """Best-effort upload of a failure screenshot to S3 so it can be viewed
    without pulling logs off the (headless, throwaway) Fargate task.

    Only runs when --s3 was requested (same opt-in as the report uploads).
    Never raises - a failed screenshot upload shouldn't mask the original
    error, so this is called from inside a try/except in the caller.
    """
```

```
    import deliver

    aws_cfg = deliver.load_aws_config(args.config)
    bucket = args.s3_bucket or aws_cfg["bucket"]
    prefix = args.s3_prefix or aws_cfg["prefix"]
    region = args.s3_region or aws_cfg["region"]
    uri = deliver.upload_error_screenshot(shot, bucket=bucket, prefix=prefix, reg
    log(f"Error screenshot available at {uri} (and {prefix}/errors/latest.png)")


def _upload_to_sharepoint(results: list[Path], args: argparse.Namespace) -> None:
    """Upload the freshly extracted files to SharePoint via the deliver module.

    Lazily imported so logix.py has no hard dependency on msal/requests unless
    --sharepoint is actually used.
    """
    try:
        import deliver
    except ImportError:
        log(
            "ERROR: --sharepoint requested but deliver/msal is unavailable. "
            "Install the SharePoint extra: uv sync --extra sharepoint"
        )
        raise

    sp_cfg = deliver.load_sharepoint_config(args.config)
    region = sp_cfg["region"] or deliver.load_aws_config(args.config)["region"]
    log(
        f"Uploading {len(results)} file(s) to SharePoint "
        f"{sp_cfg['site']} / {sp_cfg['folder']} ..."
    )
    deliver.upload_to_sharepoint(results, sp_cfg=sp_cfg, region=region)


def run(args: argparse.Namespace) -> int:
    """Run the extraction, retrying the whole browser session on failure.

    Each attempt gets a completely fresh browser context (a new ephemeral
    profile if --ephemeral-session, or the same persistent one otherwise),
    so a transient portal/network/MFA hiccup on attempt N doesn't leave
    anything around that attempt N+1 needs to clean up.
    """
    max_attempts = max(1, args.retries + 1)
    last_exc: Exception | None = None

    for attempt in range(1, max_attempts + 1):
        if attempt > 1:
```

```
            log(f"Retrying (attempt {attempt}/{max_attempts}) after {args.retry_d
            time.sleep(args.retry_delay)

        exit_code, last_exc = _run_once(args, attempt)
        if exit_code == 0:
            return 0

        if isinstance(last_exc, _NON_RETRYABLE_ERRORS):
            log(f"RESULT=FAILURE attempt={attempt} non_retryable reason={last_exc
            return exit_code

    log(f"RESULT=FAILURE attempts={max_attempts} reason={type(last_exc).__name__}
    return 1


# Failures a retry can't fix - fail fast instead of burning the retry budget.
_NON_RETRYABLE_ERRORS = (FileNotFoundError,)


def _run_once(args: argparse.Namespace, attempt: int) -> tuple[int, Exception | N
    download_dir = Path(args.download_dir).expanduser()
    credentials = load_credentials(args.config)

    with sync_playwright() as pw:
        context, page, closer = _make_context(pw, args, download_dir)

        exit_code = 0
        last_exc: Exception | None = None
        try:
            authenticate(
                page,
                args.url,
                login_timeout=args.login_timeout,
                credentials=credentials,
            )

            results: list[Path] = []
            if args.task in ("all", "oee"):
                results.append(export_oee(page, download_dir, args.year))
            if args.task in ("all", "production"):
                results.append(export_production_yield(page, download_dir))

            log("Done. Files created:")
            for path in results:
                log(f"  - {path}")

            if getattr(args, "s3", False) and results:
```

```
                _upload_to_s3(results, args)
            if getattr(args, "sharepoint", False) and results:
                _upload_to_sharepoint(results, args)
        except Exception as exc:  # noqa: BLE001
            exit_code = 1
            last_exc = exc
            log(f"ERROR (attempt {attempt}): {exc}")
            shot = download_dir / f"error_screenshot_attempt{attempt}.png"
            try:
                download_dir.mkdir(parents=True, exist_ok=True)
                page.screenshot(path=str(shot), full_page=True)
                log(f"Saved error screenshot -> {shot}")
                if getattr(args, "s3", False):
                    _upload_error_screenshot(shot, args)
            except Exception:  # noqa: BLE001
                pass
        finally:
            if args.keep_open and (args.headed or args.cdp_endpoint):
                log("Leaving the browser open (--keep-open). Press Ctrl+C to exit
                try:
                    page.wait_for_timeout(10 * 60 * 1000)
                except KeyboardInterrupt:
                    pass
            closer()
        return exit_code, last_exc


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Extract daily Tier 3 LogiX/PackOS reports for Site Neustadt.
    )
    parser.add_argument("--url", default=DEFAULT_URL, help="Portal URL to open.")
    parser.add_argument(
        "--task",
        choices=["all", "oee", "production"],
        default="all",
        help="Which export(s) to run (default: all).",
    )
    parser.add_argument(
        "--year",
        type=int,
        default=_dt.date.today().year,
        help="Reporting year for the OEE date range (default: current year).",
    )
    parser.add_argument(
        "--download-dir",
        default=str(DEFAULT_DOWNLOAD_DIR),
```

```
    help=f"Where to save exports (default: {DEFAULT_DOWNLOAD_DIR}).",
)
parser.add_argument(
    "--user-data-dir",
    default=str(DEFAULT_USER_DATA_DIR),
    help="Persistent browser profile dir (keeps you logged in).",
)
parser.add_argument(
    "--ephemeral-session",
    action="store_true",
    help=(
        "Use a fresh temporary browser profile and DELETE it on exit. Also "
        "purges any previously stored sessions at startup (the persistent "
        "profile and leftover temp profiles). Forces a clean LogiX form login
        "best for unattended/service-account runs; "
        "ignores --user-data-dir."
    ),
)
parser.add_argument(
    "--config",
    default=None,
    help=(
        "Path to a JSON credentials file (username/password) for unattended "
        "service-account sign-in. Defaults to ./config.json or the file next
        "to this script."
    ),
)
parser.add_argument(
    "--cdp-endpoint",
    default=None,
    help=(
        "Attach to an already-running browser via Chrome DevTools Protocol "
        "(e.g. http://localhost:9222). Reuses your existing logged-in Edge/"
        "Chrome session instead of launching a new browser."
    ),
)
parser.add_argument(
    "--channel",
    default=None,
    help=(
        "Launch an installed branded browser instead of bundled Chromium "
        "(e.g. 'msedge' or 'chrome'). Ignored when --cdp-endpoint is set."
    ),
)
parser.add_argument(
    "--headed",
    action="store_true",
```

```
    help="Run with a visible browser (required for the first login).",
)
parser.add_argument(
    "--keep-open",
    action="store_true",
    help="Keep the browser open after finishing (headed only).",
)
parser.add_argument(
    "--login-timeout",
    type=int,
    default=300_000,
    help="Max ms to wait for interactive sign-in (default: 300000 = 5 min).",
)
parser.add_argument(
    "--retries",
    type=int,
    default=2,
    help=(
        "Extra whole-run attempts after a failure, each with a fresh browser
        "context (default: 2, so 3 attempts total). Use 0 for the old "
        "single-attempt behavior. Meant for unattended/Fargate runs where a "
        "transient portal/network/MFA hiccup shouldn't fail the whole job."
    ),
)
parser.add_argument(
    "--retry-delay",
    type=int,
    default=30,
    help="Seconds to wait between retry attempts (default: 30).",
)
parser.add_argument(
    "--s3",
    action="store_true",
    help=(
        "After extracting, upload the downloaded files to S3 under a dated ke
        "plus a stable latest/ copy. Requires the 'aws' extra (boto3) and AWS
        "credentials (task role in AWS, or AWS_PROFILE locally)."
    ),
)
parser.add_argument(
    "--s3-bucket",
    default=None,
    help="Override the S3 bucket for --s3 (defaults to the Neustadt non-prod
)
parser.add_argument(
    "--s3-prefix",
    default=None,
```

```
        help="Override the S3 key prefix for --s3 (default: logix-report).",
    )
    parser.add_argument(
        "--s3-region",
        default=None,
        help="Override the AWS region for --s3 (default: eu-west-1).",
    )
    parser.add_argument(
        "--sharepoint",
        action="store_true",
        help=(
            "After extracting, upload the downloaded files to SharePoint (dated "
            "folder plus a stable latest/ copy) using the 'sharepoint' block in "
            "config.json. Requires the 'sharepoint' extra (boto3 + msal + request
        ),
    )
    return parser


def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    return run(args)


if __name__ == "__main__":
    sys.exit(main())
```